

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



A Report on Lab-02

QuickPOS: Incremental Development and GitHub Collaboration

Course Code: CSE 3206

Course Title: Software Engineering Sessional

Submitted By

Md. Shuvo Mia – 2203052
Mahdi Hasen Ruan – 2203053
Tanzirul Islam – 2203054

Submitted To

Emrana Kabir Hashi
Assistant Professor
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology

Date: 05/09/2026

1. Team Information

Group: 08 **Section:** A (2nd Thirty)

Name	Roll	Role	Work Done (Branch)
Tanzirul Islam	2203054	Team Lead, Backend	Git setup, Express server, Mongoose models, REST routes, React shell, Login, Products and POS pages, cash and stock logic (tanzirul/mvp-base)
Md. Shuvo Mia	2203052	Frontend Developer 1	Invoice module (<code>Invoice.jsx</code>), checkout link in <code>POS.jsx</code> , print CSS (shuvo/category-receipt)
Mahdi Hasen Ruan	2203053	Frontend Developer 2	Sales summary work, fixes in <code>saleRoutes.js</code> , backlog item B-01 (ruan/dashboard)

2. Project Overview

QuickPOS is a web based billing and stock system made for small neighborhood super shops. The idea is to replace paper bills and notebooks with one system. The cashier logs in, adds products to a cart, types the cash received and completes the sale. After that the system reduces the stock and saves the sale in the database.

This is a Minimum Viable Product (MVP), so it only covers login, products, sales, billing, stock control and a simple interface. We built it in separate modules so more features can be added later without much trouble.

3. Problem Statement

Many small shops in Bangladesh still use paper receipts and handwritten notebooks. From what we saw, this leads to a few problems:

1. Billing is slow and often wrong. Prices are looked up by hand and totals are added in the head, so mistakes happen, mostly in busy hours.
2. Stock records are not reliable. The notebook rarely matches what is actually on the shelf, so items get oversold or wasted.
3. Cash is hard to verify. Change is calculated by hand and paper records make it difficult to check the daily cash.
4. The owner gets no sales insight. It is hard to find daily revenue, best selling items or what needs to be reordered from paper records.

QuickPOS handles these with automatic totals, cash checks, live change, stock checks and saved sales records.

4. Project Objectives

- Give a quick and simple billing screen that needs very little training.
- Keep product name, price, category and stock in one place.
- Calculate the cart total automatically.

- Record the cash received, show the change live, and reject payments that are too low.
- Reduce stock after checkout and block sales that are more than the available stock.
- Save complete sale records (items, quantity, total, cash, change and time).
- Print a receipt after checkout.
- Protect the system with a staff login.
- Show sales summaries to the owner.
- Include seed data so the MVP can be tested right after setup.
- Follow incremental development using feature branches and reviewed pull requests.

5. Stakeholder Analysis

Stakeholder	Need	Main Use
Shop Owner	Daily revenue, control over products and stock, a low cost tool	Views sales, edits products, checks stock
Cashier	Fast billing and automatic change	Logs in, builds the cart, takes cash, prints the invoice
Customer	Correct bill and change, proof of purchase	Receives the printed receipt
System Admin	Manage products and user access	Manages products, resets sessions, loads seed data
Auditor (indirect)	Sales records that can be traced	Reads the saved sales history

6. Functional Requirements

ID	Requirement	Status
FR-01	Staff log in with username and password before billing.	Done
FR-02	The session stays after a page reload (localStorage) and is cleared on logout.	Done
FR-03	List all products with name, price and stock.	Done
FR-04	Add a new product with name, price and stock.	Done
FR-05	Delete a product from the catalogue.	Done
FR-06	Update product details through a REST endpoint.	API only
FR-07	Add products to the bill. Adding the same product again increases its quantity.	Done
FR-08	Clear the current bill without logging in again.	Done
FR-09	Show the running bill total.	Done
FR-10	Accept the cash given by the customer.	Done
FR-11	Show change live (Cash Received – Bill Total).	Done
FR-12	Reject checkout if cash is less than the total, on both client and server.	Done
FR-13	Reduce stock automatically after a sale.	Done
FR-14	Reject a sale if the quantity is above stock.	Done
FR-15	Save sales with items, cash, change and time.	Done

ID	Requirement	Status
FR-16	Show recent sales, newest first.	Done
FR-17	Show a printable invoice after checkout.	Done
FR-18	Provide seed data for demo and testing.	Done

7. Non-Functional Requirements

Category	Requirement	ID
Performance	Checkout should finish within 2 seconds on a local network. The stock view should refresh without reloading the whole page.	NFR-01, 02
Usability	Billing can be done from one screen using keyboard and mouse. Delete buttons look different from other buttons.	NFR-03, 04
Portability	Runs offline on a single shop computer, in any standard browser.	NFR-05, 06
Maintainability	Frontend, backend, routes and models are kept separate. One <code>api.js</code> file handles all HTTP calls.	NFR-07, 08
Security	Every screen needs a login. The payment is checked again on the server.	NFR-09, 10
Reliability	The sale is saved before the cart is cleared. Stock never goes negative. An invalid sale fails completely.	NFR-11, 12
Scalability and Print	Should work with up to 10,000 products. The invoice has its own <code>@media print</code> style.	NFR-13, 14

8. User Stories and Acceptance Criteria

US-1 Staff Login

As a cashier, I want to log in so that the billing screen is protected.

- A valid login (admin / 1234) opens the billing page and stays open after a reload.
- A wrong login is denied.
- Logout ends the session and goes back to the login screen.

US-2 Build a Bill

As a cashier, I want to add products and see the total so that I can serve customers fast.

- Pressing Add puts the product in the cart with quantity 1 and updates the total.
- Pressing Add again increases the quantity instead of making a duplicate line.

US-3 Cash and Change

As a cashier, I want the change calculated automatically so that I don't make mistakes in my head.

- Bill 490 Tk, cash 500 Tk: change of 10 Tk is shown and checkout is enabled.
- Bill 490 Tk, cash 400 Tk: "insufficient payment" is shown and checkout is disabled.

US-4 Stock Control

As a shop owner, I want stock to go down after checkout so that my records stay correct.

- Stock is 100 and 2 are sold: stock becomes 98.
- Stock is 1 and 5 are requested: the sale is rejected and stock stays 1.

US-5 Printable Invoice

As a cashier, I want a printable invoice so that the customer has proof of purchase.

- After checkout the modal shows items, subtotal, cash paid, change and grand total.
- Print Invoice opens the print dialog and prints only the receipt.

US-6 Sales History

As a shop owner, I want to see past sales so that I can keep track of revenue.

- The Sales page lists sales newest first with time, items, total, cash and change.

9. Selected Process Model: Incremental Development

We built QuickPOS in four increments. Each one added a working part and was merged into `main` through a reviewed pull request.

Increment	What Was Delivered	Branch / Lead
1. POS Core Baseline	App shell with login, catalogue, cart, cash handler, REST API, MongoDB and sales history	<code>tanzirul/mvp-base</code> , Tanzirul Islam
2. Sales Summary	Route level fixes. The summary widget was moved to backlog B-01	<code>ruan/dashboard</code> , Mahdi Hasan Ruan
3. Printable Invoice	Invoice modal and print stylesheet	<code>shuvo/category-receipt</code> , Shuvo Mia
4. Report and Hardening	Design report, workflow notes and screenshots	<code>tanzirul/report-lab2</code> , Tanzirul Islam

10. Why Incremental Development?

Reason	Explanation
Fits the MVP scope	We could start with a small working core and add to it step by step.
Easy to split	POS features break down into independent parts that can be demonstrated separately.
Matches our Git workflow	One branch, one PR and one merge per increment.
Parallel work	All three of us could work at the same time with few merge conflicts.
Low risk	If one increment broke, the merged baseline was not affected.

11. Comparison with Other Models

Criterion	Incremental (chosen)	Waterfall	Spiral	Scrum
Stable requirements needed	Medium	High	Medium	Low to medium
Feedback	Every increment	Mostly at the end	Every loop	Every sprint
First demo	Days	Weeks or months	Fairly long	Days
Documentation	Light	Heavy	Heavy (risk)	Medium
Fit for a 3 member lab	Strong	Weak	Weak	Moderate to strong

Waterfall needs frozen requirements and delivers late. Some features, like the invoice, were added after the core was ready, so it did not suit us. Spiral adds formal risk cycles, but our stack (React, Express, MongoDB) is low risk and lab time is short, so it would only be extra work. Scrum with sprint planning, reviews and retrospectives is too heavy for three people on one lab project.

In the end, Incremental Development gave us the best balance of a simple workflow, steady delivery and control over the team's work.

12. MVP Design Overview

12.1 Technology Stack

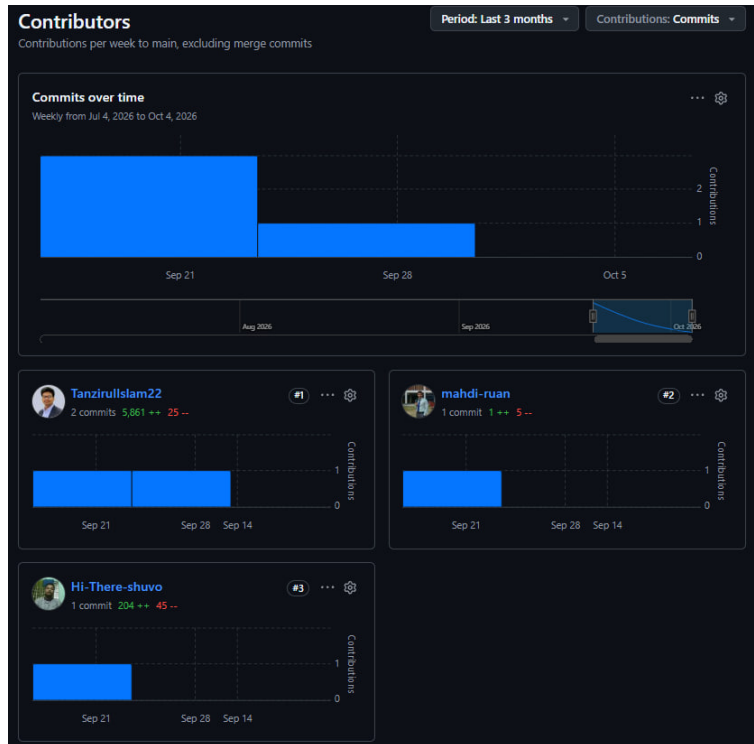
- **Frontend:** React 18.3 and Vite 5.4.
- **Backend:** Node.js and Express 4.19 (REST API).
- **Database:** MongoDB with Mongoose 8.5 (Product and Sale data).
- **Styling:** Tailwind CSS 3.4.
- **State:** React page state in `App.jsx`, and `localStorage` for the session.
- **API helper:** `src/api.js` for all HTTP calls.
- **Report:** LaTeX compiled with `pdflatex`.

12.2 UI and Server Components

Component	Responsibility
<code>App.jsx</code>	App shell, session gate, page navigation and login state in <code>localStorage</code> .
<code>Login.jsx</code>	Staff login screen (admin / 1234).
<code>Products.jsx</code>	List, add and delete products.
<code>POS.jsx</code>	Cart, running total, cash input, live change and checkout.
<code>Sales.jsx</code>	Recent sales with time, items, total, paid and change.
<code>Invoice.jsx</code>	Printable receipt modal with an <code>@media print</code> style.
<code>api.js</code>	Base URL, <code>getJSON</code> and <code>postJSON</code> .
<code>server.js</code>	Starts the server, connects to the database and serves <code>/api/seed</code> demo data.

13. GitHub Collaboration

Repository: <https://github.com/TanzirulIslam22/QuickPOS>



13.1 Directory Structure

```
QuickPOS/
|-- client/                                React 18 + Vite frontend
|   |-- src/
|       |-- api.js                        API fetch helper
|       |-- App.jsx                      Session gate and navigation
|       |-- pages/                       Login, Products, POS, Sales, Invoice
|-- server/                               Express + MongoDB backend
|   |-- server.js                        Bootstrap and seed API
|   |-- models/                         Product.js, Sale.js
|   |-- routes/                         productRoutes.js, saleRoutes.js
|-- docs/                                Requirement report, TEAM_WORKFLOW.md
|-- screenshots/                         MVP screenshots
|-- README.md                           Overview and API notes
```

13.2 Branching Strategy

Branch	Owner	Purpose
main (protected)	All members	Stable, reviewed code
tanzirul/mvp-base	Tanzirul Islam	Increment 1: core POS and API
ruan/dashboard	Mahdi Hasan Ruan	Increment 2: route guards and sales summary
shuvo/category-receipt	Shuvo Mia	Increment 3: invoice and print style
tanzirul/report-lab2	Tanzirul Islam	Increment 4: report and screenshots

13.3 Pull Request Rules

1. Create every feature branch from the latest `main` (`git pull origin main`).
2. Each PR should say which features were delivered, which requirements are met and what is left in the backlog.
3. At least one other member has to review and approve before the merge.
4. The author fixes merge conflicts locally and tests again before asking for another review.
5. Nobody pushes directly to `main`. Everything goes through a reviewed PR.

14. Challenges and Solutions

Challenge	Problem	Solution
C-1: Payment check could be bypassed	A user could edit the page in the browser and complete a sale with too little cash.	<code>saleRoutes.js</code> now recalculates the total on the server and returns HTTP 400 if the cash is low.
C-2: Partial stock reduction	If a later item had no stock, the earlier items could already be reduced.	Done in two steps. First check the stock of all items, then reduce stock and save the sale.
C-3: Bloated billing component	Receipt and print code made <code>POS.jsx</code> large and messy.	Moved the receipt code to <code>Invoice.jsx</code> . Now <code>POS.jsx</code> only handles the cart and checkout.
C-4: Scope drift	PR #2 was opened for a full dashboard, but only the route security got merged.	We corrected the report to match the merged code and listed the widget as backlog B-01.

15. Conclusion

This Lab 2 delivered the first MVP of QuickPOS. It replaces manual cash memos and handwritten stock books. It meets 18 functional and 14 non-functional requirements, except that FR-06 is available through the API only and the sales summary widget is still in backlog B-01.

The team used Incremental Development with four branches and reviewed pull requests, so the three of us could work separately while `main` stayed stable. The stack of React, Vite, Tailwind, Node.js, Express and MongoDB is modular and gives a good base for future lab work like design patterns, automated testing and more features.